CampusEats — SOAP Partner Integration

Team

Team Id: Members name : Anshu Mala 20252651010 (Leader) Sanika Jain 20252651046 Annu Mishra 20252651009 Mritunjay Maurya 20252651037

Context

CampusEats integrates with **PayFlex**, an external payment gateway, to process student payments for orders. We integrate the `charge` operation via SOAP, even though the rest of CampusEats is built on REST, for three reasons.

First, payment processing needs a **strong, formal contract** — the gateway publishes a WSDL that precisely defines every field, type, and fault a client must handle, which matters far more here than for a simple resource lookup. Second, financial transactions require **message-level security**: SOAP's WS-Security standard lets us sign and encrypt the payment payload itself (not just the transport), so the message stays protected even if it passes through intermediaries. Third, payment gateways commonly need to guarantee **transactional reliability** — knowing definitively whether a charge succeeded, failed, or is still pending — which SOAP's mature WS-* standards (WS-ReliableMessaging, WS-AtomicTransaction) are built to support, unlike plain REST/HTTP.

The operation we integrate: **charge(orderId, amount, cardToken) → paymentId, status**

HTTP binding

The SOAP request is carried as an HTTP POST to the endpoint defined in the WSDL's service/port section. Below is the raw HTTP request block:

```
POST /soap/payment HTTP/1.1
Host: api.payflex.example.com
Content-Type: text/xml; charset=utf-8
Content-Length: 512
SOAPAction: "http://payflex.example.com/payment/charge"

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:pay="http://payflex.example.com/payment/wsdl">
  <soap:Header>
    <pay:Credentials>
      <pay:apiKey>CAMPUSEATS_9F3A21B7</pay:apiKey>
      <pay:merchantId>CAMPUSEATS_UNIV01</pay:merchantId>
    </pay:Credentials>
  </soap:Header>
  <soap:Body>
    <pay:ChargeRequest>
      <pay:orderId>ORD-58213</pay:orderId>
      <pay:amount>249.00</pay:amount>
```

```
        <pay:currency>INR</pay:currency>
        <pay:cardToken>tok_4f8e2a9c7b1d</pay:cardToken>
      </pay:ChargeRequest>
    </soap:Body>
  </soap:Envelope>
```

Discovery

CampusEats obtains the PayFlex WSDL through a **direct URL**, published by PayFlex on their developer portal, rather than a live UDDI registry (UDDI is largely obsolete in modern practice). The WSDL is fetched once at integration time and stored alongside our own contracts in `partner.wsdl`, so our Payment Service does not depend on PayFlex's portal being reachable at runtime.

For internal tracking, CampusEats also maintains a small **service catalogue** — a lightweight internal registry recording every external partner we integrate with, similar in spirit to a UDDI tModel entry but stored as a simple document/database table rather than a live registry server.

Catalogue entry

Field	Value
Business name	PayFlex Inc.
Service name	PayFlex Payment Service
Endpoint	https://api.payflex.example.com/soap/payment
WSDL location	https://developer.payflex.example.com/docs/wsdl/partner.wsdl
Operation used	charge
Owning CampusEats service	Payment Service

Fault mapping

PayFlex's SOAP fault uses its own vocabulary (`card_declined`, `gateway_timeout`, etc.), which must not leak into CampusEats' own contract. The Payment Service acts as a translation layer: it catches the partner's `soap:Fault`, inspects the `faultCode`, and maps it to the error CampusEats' own `placeOrder` contract (Assignment 2, Task 4) already promises callers.

PayFlex fault code	Meaning (PayFlex vocabulary)	Mapped to (CampusEats contract)
`card_declined`	The card issuer declined the transaction	`PaymentFailed`
`gateway_timeout`	PayFlex did not respond in time	`PaymentFailed`
`invalid_card_token`	The card token was malformed or expired	`PaymentFailed`
`insufficient_funds`	The card lacks funds for this amount	`PaymentFailed`

CampusEats students only ever see `PaymentFailed` (with a generic, student-friendly message like "Payment could not be processed, please try again"). The specific PayFlex fault code is logged internally for debugging

and support purposes, but it is never returned in the `placeOrder` response — this keeps our contract stable even if we later switch payment providers.